

Refatoração SOLID, Clean Code e Extensibilidade

Disciplina: Padrões e Arquitetura de Software

Alunos: João Pedro Zangerolamo de Freitas, Caiuã Soares Talli Vieira, Felipe Corso Pretoni, Gustavo Albiero Zorzo, Victor Reis de Palma, Mateus Giroto Mergulhão, Marcelo França Michelino de Oliveira

Repositório: <https://github.com/felipecorsopretoni/Padroes-Arquitetura-de-Software>

Data: 26/05/2026

1. (a) Identificação das Violações SOLID

1.1 SRP — Single Responsibility Principle

- legacy.py concentrava criação de pedidos, persistência, notificação, pagamento e relatórios.

Linhas: 6-9, 15, 90, 116, 141

Arquivo: Código legado

Justificativa técnica:

A classe Sis possui responsabilidades de várias camadas do sistema ao mesmo tempo. Ela cria pedidos, calcula valores, salva dados no SQLite, envia notificações via print, processa pagamentos, gera relatórios e valida estoque. Isso viola o SRP porque a classe tem muitos motivos para mudar: uma alteração na regra de desconto, no banco de dados, no formato do relatório ou no envio de notificações obrigaria mudanças na mesma classe.

Impacto prático:

Qualquer mudança futura ficaria cara e arriscada. Por exemplo, trocar o SQLite por outro banco, adicionar envio real de e-mail, mudar regras de pagamento ou alterar o

relatório poderia quebrar partes não relacionadas do sistema. Além disso, testes unitários ficam difíceis, porque para testar pedido também se acaba envolvendo banco, relatório, notificação e pagamento.

- Regras de fidelidade e transições de status estavam misturadas no mesmo fluxo.

Linhas: 59-80

Arquivo: Código legado

Justificativa técnica:

O método `upd_st` não apenas altera o status do pedido, mas também executa notificações e calcula pontos de fidelidade. Essas são responsabilidades diferentes: controle de estado do pedido, comunicação com o cliente e regra de pontuação. Ao misturar tudo no mesmo método, o código perde coesão e se torna dependente de detalhes que deveriam estar em serviços separados.

Impacto prático:

Alterações futuras em regras de fidelidade ou em mensagens de notificação exigiriam modificar o método de atualização de status. Por exemplo, se clientes VIP passarem a ganhar 3 vezes mais pontos, ou se pedidos entregues também precisarem gerar SMS, seria necessário mexer em um fluxo crítico de status, aumentando o risco de bugs em aprovações, envios e entregas.

1.2 OCP — Open/Closed Principle

- Cadeias de `if/elif` controlavam descontos por item e meios de pagamento.

Linhas: 18-26, 123-139

Arquivo: Código legado

Justificativa técnica:

O código usa condicionais para decidir o comportamento com base em valores como tipo do item ou método de pagamento. Isso viola o OCP porque, sempre que surge um novo tipo de desconto ou pagamento, é necessário modificar métodos já existentes, como `add_ped` ou `proc_pag`. O ideal seria adicionar novas classes ou estratégias sem alterar o código central.

Impacto prático:

Adicionar novas formas de pagamento, como PayPal, cartão-presente ou criptomoeda, exigiria editar o método `proc_pag`. Da mesma forma, criar novos descontos, como `desc30`, “compre 2 leve 3” ou cupom promocional, exigiria alterar `add_ped`. Isso aumenta o risco de regressão em regras antigas que já funcionavam.

- Novos canais de notificação exigiriam alterar a implementação central.

Linhas: 39-47, 64-71

Arquivo: Código legado

Justificativa técnica:

A lógica de notificação está embutida diretamente nos métodos de pedido e status, usando condicionais para decidir se envia e-mail, SMS ou notificação ao gerente. Isso viola o OCP porque o comportamento não é extensível por meio de abstrações. Para adicionar WhatsApp, push notification ou integração com Slack, seria necessário modificar a classe principal.

Impacto prático:

Qualquer novo canal de comunicação exigiria mexer no fluxo central de criação ou atualização de pedidos. Isso torna mudanças simples mais perigosas. Por exemplo, adicionar WhatsApp apenas para clientes VIP poderia acabar afetando pedidos normais ou corporativos por erro nas condicionais.

1.3 LSP — Liskov Substitution Principle

- PedidoEspecial sobrescrevia comportamento sem manter as invariantes da classe base.

Linhas: 18-24, 163-181

Arquivo: Código legado

Justificativa técnica:

PedEspecial herda de Sis, mas redefine add_ped com regras diferentes. A classe base calcula descontos conforme o tipo do item e aplica regras de cliente. Já PedEspecial altera o cálculo com um acréscimo fixo de 15% e muda o comportamento de notificação. Isso quebra o LSP porque um objeto PedEspecial não se comporta de forma substituível em relação a Sis.

Impacto prático:

Um código que espera receber uma instância de Sis poderia funcionar incorretamente ao receber PedEspecial. Por exemplo, relatórios, cálculos ou fluxos de pedido poderiam gerar valores inesperados, porque a subclasse altera regras fundamentais da classe base. Isso torna a herança perigosa e dificulta criar novos tipos de pedido.

- A hierarquia foi eliminada por composição para evitar substituição incorreta.

Linhas: 183-190

Arquivo: Código legado

Justificativa técnica:

A subclasse sobrescreve upd_st permitindo atualizar diretamente para qualquer status, ignorando possíveis validações ou transições intermediárias esperadas pela classe base. Isso viola o LSP porque a subclasse enfraquece o contrato comportamental da

superclasse. Se a classe base pressupõe um fluxo como pendente - aprovado - enviado - entregue, a subclasse não deveria permitir quebrar essa sequência.

Impacto prático:

Pedidos especiais poderiam entrar em estados inválidos, como ir diretamente de pendente para entregue, sem passar por aprovação ou envio. Isso compromete relatórios, notificações, pagamentos e regras de negócio que dependem de um ciclo de vida consistente do pedido.

1.4 ISP — Interface Segregation Principle

- A fachada antiga expunha várias responsabilidades sem contratos específicos.

Linhas: 15, 51, 59, 82, 90, 116, 141, 153, 159

Arquivo: Código legado

Justificativa técnica:

A classe `Sis` funciona como uma fachada grande demais. Ela oferece métodos de pedido, pagamento, relatório, estoque, cancelamento, consulta e conexão com banco. Mesmo sem existir uma interface formal em Python, a classe cria um contrato amplo e genérico demais para seus consumidores. Isso viola o ISP porque clientes que precisam apenas de uma operação acabam dependendo de uma classe com muitas operações não relacionadas.

Impacto prático:

Um módulo que só precisa gerar relatórios passa a depender também de métodos de pagamento, estoque e criação de pedidos. Isso aumenta o acoplamento e dificulta manutenção. Se a assinatura de um método de pagamento mudar, por exemplo, consumidores interessados apenas em relatório ainda podem ser afetados por dependerem da mesma fachada.

- O repositório não era separado das demais colaborações.

Linhas: 8-13, 34-37, 52-56, 91-112, 155-156

Arquivo: Código legado

Justificativa técnica:

O acesso ao banco de dados está misturado com regras de negócio dentro da classe Sis. Não existe um contrato específico para persistência, como um OrderRepository. Assim, os métodos de aplicação dependem diretamente de comandos SQL e da conexão SQLite. Isso fere o ISP porque não há interfaces pequenas e específicas separando persistência, regra de negócio, relatório e pagamento.

Impacto prático:

Trocar a forma de armazenamento ficaria caro. Por exemplo, mudar de SQLite para PostgreSQL, arquivo JSON ou API externa exigiria alterar vários métodos da classe principal. Além disso, testar regras de negócio sem banco real fica mais difícil, porque a persistência está acoplada ao restante do sistema.

1.5 DIP — Dependency Inversion Principle

- Os módulos de aplicação criavam objetos concretos diretamente.

Linhas: 163-164, 194

Arquivo: Código legado

Justificativa técnica:

A classe Sis cria diretamente a conexão com SQLite usando `sqlite3.connect('loja.db')`. Isso faz com que a regra de alto nível dependa de um detalhe concreto de infraestrutura. O DIP recomenda que módulos de alto nível dependam de abstrações, não de implementações específicas. O banco deveria ser acessado por meio de uma interface ou repositório abstrato.

Impacto prático:

Trocar o banco de dados ou simular persistência em testes exigiria alterar o código da classe principal. Por exemplo, usar um banco em memória para testes, migrar para PostgreSQL ou salvar os pedidos em outro serviço seria difícil porque a dependência concreta está instanciada diretamente dentro da classe.

- Regras de negócio dependiam de detalhes de notificação e pagamento.

Linhas: 40-47, 64-71, 123-139

Arquivo: Código legado

Justificativa técnica:

As regras de pedido chamam diretamente detalhes concretos, como print para simular e-mail, SMS e notificações. O mesmo ocorre no pagamento, onde o método `proc_pag` conhece diretamente as etapas de cartão, Pix e boleto. Isso viola o DIP porque a lógica principal depende de implementações concretas, em vez de depender de abstrações como `NotificationObserver`, `NotificationPublisher` ou `PaymentStrategy`.

Impacto prático:

Substituir print por envio real de e-mail, SMS, WhatsApp ou fila de mensagens exigiria alterar as regras de negócio. Da mesma forma, trocar ou adicionar meios de pagamento exigiria modificar o fluxo principal. Isso torna o sistema rígido, difícil de testar e arriscado para evoluir.

2. (b) Soluções Implementadas

2.1 SRP — legacy.py concentrava criação de pedidos, persistência, notificação, pagamento e relatórios.

Solução adotada:

A classe Sis deixou de implementar diretamente todas as responsabilidades e passou a atuar como uma fachada de compatibilidade, delegando as operações para serviços especializados.

A persistência foi isolada em SQLiteOrderRepository, os pedidos em OrderService, os pagamentos em PaymentService e os relatórios em ReportService.

Padrão GoF utilizado:

Strategy, Observer

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Classe	OrderService	Centralizar regras de criação, consulta, status, estoque e cancelamento de pedidos.
Classe	PaymentService	Processar pagamentos.
Classe	ReportService	Gerar relatórios e cálculos por cliente.
Classe	SQLiteOrderRepository	Persistir e consultar pedidos no SQLite.

Interface	OrderRepositoryInterface	Abstrair operações de persistência de pedidos.

Antes:

```
class Sis:
```

```
    def __init__(self):
```

```
        self.db = sqlite3.connect('loja.db')
```

```
        self.c = self.db.cursor()
```

```
    def add_ped(self, n, its, t):
```

```
        dt = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
```

```
        tot = 0
```

```
        for i in its:
```

```
            if i['tipo'] == 'normal':
```

```
                tot += i['p'] * i['q']
```

```
            elif i['tipo'] == 'desc10':
```

```
                tot += i['p'] * i['q'] * 0.9
```

```
            elif i['tipo'] == 'desc20':
```

```
                tot += i['p'] * i['q'] * 0.8
```

```
        its_str = json.dumps(its)
```

```
        self.c.execute(
```

```
            "INSERT INTO ped (cli, itens, tot, st, dt, tp) VALUES (?, ?, ?, ?, ?, ?)",
```

```
            (n, its_str, tot, 'pendente', dt, t)
```

```
)

self.db.commit()

def gerar_rel(self, tipo):

    if tipo == 'vendas':

        self.c.execute("SELECT * FROM ped")

        # lógica de relatório
```

```
def proc_pag(self, id, m, vl):

    if m == 'cartao':

        print("Processando pagamento com cartao...")

    elif m == 'pix':

        print("Gerando QR Code PIX...")
```

Depois:

```
class Sis:

    def __init__(self, db_path: str = "loja.db") -> None:

        container = build_app_container(db_path)

        self._repository = container.repository

        self._order_service = container.order_service

        self._payment_service = container.payment_service

        self._report_service = container.report_service

    def add_ped(self, n: str, its: list[LegacyItemPayload], t: str) -> int:

        return self._order_service.create_order(n, its, t)
```

```

def gerar_rel(self, tipo: str) -> None:

    self._report_service.generate(tipo)


def proc_pag(self, order_id: int, method: str, value: float) -> bool:
    order = self._order_service.get_order_entity(order_id)
    if order is None:
        print("Metodo de pagamento invalido!")
        return False

    return self._payment_service.process_payment(
        order_id=order_id,
        method=method,
        value=value,
        order_total=order.total,
    )

```

2.2 SRP — Regras de fidelidade e transições de status estavam misturadas no mesmo fluxo.

Solução adotada:

A atualização de status foi delegada ao OrderService, enquanto notificações passaram a ser publicadas por um serviço próprio e distribuídas para observers especializados.

As regras de pontuação e ajustes também deixam de ficar diretamente dentro do método de atualização de status.

Padrão GoF utilizado:

Observer, Strategy

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
------	------	------------------

Classe	OrderService	Atualizar status e coordenar regras de pedido.
Classe	NotificationService	Disparar notificações.
Classe	NotificationPublisher	Publicar eventos para observers.
Classe abstrata/interface	NotificationObserver	Contrato para notificações especializadas.
Classe abstrata/interface	OrderAdjustmentStrategy	Contrato para ajustes aplicáveis ao pedido.

Antes:

```
def upd_st(self, id, s):

    p = self.get_ped(id)

    if p:

        self.c.execute("UPDATE ped SET st=? WHERE id=?", (s, id))

        self.db.commit()

    if s == 'aprovado':

        print(f"Email enviado para {p['cli']}: Pedido aprovado!")

    if p['tp'] == 'vip':

        print(f"SMS enviado para {p['cli']}: Pedido aprovado!")

    elif s == 'enviado':

        print(f"Email enviado para {p['cli']}: Pedido enviado!")

    elif s == 'entregue':

        print(f"Email enviado para {p['cli']}: Pedido entregue!")
```

```

if p['tp'] == 'vip':

    pts = int(p['tot'] * 2)

    print(f"Cliente VIP ganhou {pts} pontos!")

elif p['tp'] == 'corporativo':

    pts = int(p['tot'] * 1.5)

    print(f"Cliente corporativo ganhou {pts} pontos!")

else:

    pts = int(p['tot'])

    print(f"Cliente ganhou {pts} pontos!")

```

Depois:

```

def upd_st(self, order_id: int, status: str) -> None:

    self._order_service.update_status(order_id, status)

```

2.3 OCP — Cadeias de if/elif controlavam descontos por item e meios de pagamento.

Solução adotada:

As regras de desconto e pagamento foram substituídas por estratégias plugáveis. Assim, novos tipos de desconto ou pagamento podem ser adicionados criando novas classes, sem alterar o código central.

Padrão GoF utilizado:

Strategy, Factory Method

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Classe abstrata/interface	ItemPricingStrategy	Definir regra de preço por tipo de item.
Classe abstrata/interface	CustomerDiscountStrategy	Definir desconto por tipo de cliente.

Classe abstrata/interface	OrderAdjustmentStrategy	Definir ajustes no pedido.
Classe abstrata/interface	PaymentStrategy	Definir contrato para meios de pagamento.
Classe	PricingService	Aplicar estratégias de preço e desconto.
Classe	PaymentStrategyRegistry	Registrar e localizar estratégias de pagamento.
Classe	PaymentService	Processar pagamento usando a estratégia correta.
Classe	OrderFactoryRegistry	Criar pedidos conforme o tipo.

Antes:

for i in its:

if i['tipo'] == 'normal':

tot += i['p'] * i['q']

elif i['tipo'] == 'desc10':

tot += i['p'] * i['q'] * 0.9

elif i['tipo'] == 'desc20':

tot += i['p'] * i['q'] * 0.8

elif i['tipo'] == 'frete_gratis':

tot += i['p'] * i['q']

def proc_pag(self, id, m, vl):

p = self.get_ped(id)

```
if vl < p['tot']:
```

```
    print("Valor insuficiente!")
```

```
    return False
```

```
if m == 'cartao':
```

```
    print("Processando pagamento com cartao...")
```

```
    print("Cartao validado!")
```

```
    self.upd_st(id, 'aprovado')
```

```
    return True
```

```
elif m == 'pix':
```

```
    print("Gerando QR Code PIX...")
```

```
    print("PIX recebido!")
```

```
    self.upd_st(id, 'aprovado')
```

```
    return True
```

```
elif m == 'boleto':
```

```
    print("Gerando boleto...")
```

```
    print("Boleto gerado!")
```

```
    return True
```

```
else:
```

```
    print("Metodo de pagamento invalido!")
```

```
    return False
```

Depois:

```
def add_ped(self, n: str, its: list[LegacyItemPayload], t: str) -> int:
```

```
    return self._order_service.create_order(n, its, t)
```

```
def proc_pag(self, order_id: int, method: str, value: float) -> bool:
```

```

order = self._order_service.get_order_entity(order_id)

if order is None:

    print("Metodo de pagamento invalido!")

    return False


return self._payment_service.process_payment(

    order_id=order_id,

    method=method,

    value=value,

    order_total=order.total,

)

```

2.4 OCP — Novos canais de notificação exigiriam alterar a implementação central.

Solução adotada:

As notificações foram separadas usando NotificationService, NotificationPublisher e NotificationObserver.

Agora, novos canais podem ser adicionados criando novos observers, sem alterar o fluxo principal de criação ou atualização de pedidos.

Padrão GoF utilizado:

Observer

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Classe	NotificationService	Coordenar notificações do sistema.
Classe	NotificationPublisher	Publicar eventos para os observers.

Classe abstrata/interface	NotificationObserver	Definir contrato para canais de notificação.
---------------------------	----------------------	--

Antes:

```

if t == 'normal':

    print(f"Email enviado para {n}: Pedido recebido!")

elif t == 'vip':

    print(f"Email enviado para {n}: Pedido recebido!")

    print(f"SMS enviado para {n}: Pedido VIP recebido!")

elif t == 'corporativo':

    print(f"Email enviado para {n}: Pedido recebido!")

    print(f"Notificacao enviada ao gerente de conta de {n}")


if s == 'aprovado':

    print(f"Email enviado para {p['cli']}: Pedido aprovado!")

    if p['tp'] == 'vip':

        print(f"SMS enviado para {p['cli']}: Pedido aprovado!")

elif s == 'enviado':

    print(f"Email enviado para {p['cli']}: Pedido enviado!")

elif s == 'entregue':

    print(f"Email enviado para {p['cli']}: Pedido entregue!")

```

Depois:

```

def add_ped(self, n: str, its: list[LegacyItemPayload], t: str) -> int:

    return self._order_service.create_order(n, its, t)


def upd_st(self, order_id: int, status: str) -> None:

```

```
self._order_service.update_status(order_id, status)
```

2.5 LSP — PedidoEspecial sobrescrevia comportamento sem manter as invariantes da classe base.

Solução adotada:

PedEspecial deixou de herdar de Sis e passou a usar composição. Em vez de reaproveitar a classe base por herança, ela monta um container próprio com build_special_app_container.

Padrão GoF utilizado:

Factory Method, Strategy

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Classe	PedEspecial	Fachada para pedidos especiais usando composição.
Classe	SpecialOrderService	Serviço específico para comportamento de pedido especial.
Classe	OrderFactoryRegistry	Resolver criação de pedidos por tipo.
Classe abstrata/interface	OrderAdjustmentStrategy	Permitir ajuste especial sem herança problemática.

Antes:

```
class PedEspecial(Sis):  
  
    def add_ped(self, n, its, t):  
  
        dt = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
```

```
tot = 0
```

```
for i in its:
```

```
    if i['tipo'] == 'normal':
```

```
        tot += i['p'] * i['q']
```

```
    elif i['tipo'] == 'desc10':
```

```
        tot += i['p'] * i['q'] * 0.9
```

```
    elif i['tipo'] == 'desc20':
```

```
        tot += i['p'] * i['q'] * 0.8
```

```
tot = tot * 1.15
```

```
its_str = json.dumps(its)
```

```
self.c.execute(
```

```
    "INSERT INTO ped (cli, itens, tot, st, dt, tp) VALUES (?, ?, ?, ?, ?, ?)",
```

```
    (n, its_str, tot, 'pendente', dt, t)
```

```
)
```

```
self.db.commit()
```

```
print(f"Email especial enviado para {n}: Pedido especial recebido!")
```

```
return self.c.lastrowid
```

Depois:

```
class PedEspecial:
```

```
    def __init__(self, db_path: str = "loja.db") -> None:
```

```
        container = build_special_app_container(db_path)
```

```
        self._repository = container.repository
```

```
self._order_service = container.order_service
```

```
def add_ped(self, n: str, its: list[LegacyItemPayload], t: str) -> int:
```

```
    return self._order_service.create_order(n, its, t)
```

2.6 LSP — A hierarquia foi eliminada por composição para evitar substituição incorreta.

Solução adotada:

A relação de herança entre PedEspecial e Sis foi removida. No código legado, PedEspecial herdava de Sis e sobrescrevia métodos importantes, como add_ped e upd_st, alterando o comportamento esperado da classe base.

Na versão refatorada, PedEspecial não é mais uma subclasse de Sis. Em vez disso, ela usa composição: cria seu próprio container com build_special_app_container e delega as operações para um serviço específico.

Padrão GoF utilizado:

Nenhum

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Classe	PedEspecial	Fachada de pedidos especiais.
Classe	SpecialOrderService	Serviço específico para pedidos especiais.

Antes:

```
class PedEspecial(Sis):
```

```
    def upd_st(self, id, s):
```

```
        # PedEspecial pula direto para qualquer estado
```

```
# ignorando transicoes intermediarias do pai

p = self.get_ped(id)

if p:

    self.c.execute("UPDATE ped SET st=? WHERE id=?", (s, id))

    self.db.commit()

    print(f"Pedido especial {id} -> {s}")
```

Depois:

```
class PedEspecial:

    def __init__(self, db_path: str = "loja.db") -> None:

        container = build_special_app_container(db_path)

        self._repository = container.repository

        self._order_service = container.order_service

    def upd_st(self, order_id: int, status: str) -> None:

        self._order_service.update_status(order_id, status)
```

2.7 ISP — A fachada antiga expunha várias responsabilidades sem contratos específicos.

Solução adotada:

As responsabilidades foram separadas em serviços e contratos menores. A fachada Sis ainda existe para compatibilidade, mas agora ela apenas delega para serviços especializados.

Padrão GoF utilizado:

Strategy, Observer

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
------	------	------------------

Classe	OrderService	Operações relacionadas a pedidos.
Classe	PaymentService	Operações relacionadas a pagamento.
Classe	ReportService	Operações relacionadas a relatórios.
Interface	OrderRepositoryInterface	Contrato de persistência.
Classe abstrata/interface	PaymentStrategy	Contrato de pagamento.
Classe abstrata/interface	NotificationObserver	Contrato de notificação.

Antes:

```
class Sis:
```

```
    def add_ped(self, n, its, t):
```

```
    ...
```

```
    def get_ped(self, id):
```

```
    ...
```

```
    def upd_st(self, id, s):
```

```
    ...
```

```
    def calc_tot_cli(self, n):
```

```
    ...
```

```
    def gerar_rel(self, tipo):
```

```
    ...
```

```
def proc_pag(self, id, m, vl):
```

```
...
```

```
def validar_estoque(self, its):
```

```
...
```

```
def cancelar_pedido(self, id):
```

```
...
```

```
def close(self):
```

```
...
```

Depois:

```
class Sis:
```

```
def __init__(self, db_path: str = "loja.db") -> None:
```

```
    container = build_app_container(db_path)
```

```
    self._repository = container.repository
```

```
    self._order_service = container.order_service
```

```
    self._payment_service = container.payment_service
```

```
    self._report_service = container.report_service
```

```
def add_ped(self, n: str, its: list[LegacyItemPayload], t: str) -> int:
```

```
    return self._order_service.create_order(n, its, t)
```

```
def gerar_rel(self, tipo: str) -> None:
```

```
    self._report_service.generate(tipo)
```

```
def proc_pag(self, order_id: int, method: str, value: float) -> bool:
```

```
...
```

```
return self._payment_service.process_payment(...)
```

2.8 ISP — O repositório não era separado das demais colaborações.

Solução adotada:

O acesso ao banco de dados foi movido para `SQLiteOrderRepository`, que implementa `OrderRepositoryInterface`.

Os serviços deixam de executar SQL diretamente e passam a depender do contrato de repositório.

Padrão GoF utilizado:

Nenhum

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Interface	<code>OrderRepositoryInterface</code>	Definir operações necessárias para persistência de pedidos.
Classe	<code>SQLiteOrderRepository</code>	Implementar persistência usando SQLite.

Antes:

```
self.db = sqlite3.connect('loja.db')
```

```
self.c = self.db.cursor()
```

```
self.c.execute(
```

```
"INSERT INTO ped (cli, itens, tot, st, dt, tp) VALUES (?, ?, ?, ?, ?, ?)",
```



```
(n, its_str, tot, 'pendente', dt, t)
```

```
)
```

```
self.db.commit()
```

```
self.c.execute("SELECT * FROM ped WHERE id=?", (id,))
```

```
self.c.execute("SELECT * FROM ped")
```

Depois:

```
class Sis:
```

```
def __init__(self, db_path: str = "loja.db") -> None:
```

```
    container = build_app_container(db_path)
```

```
    self._repository = container.repository
```

```
def close(self) -> None:
```

```
    self._repository.close()
```

2.9 DIP — Os módulos de aplicação criavam objetos concretos diretamente.

Solução adotada:

A criação dos objetos concretos foi concentrada nas funções de composição `build_app_container` e `build_special_app_container`.

Os serviços recebem suas dependências prontas por meio do container, em vez de criarem objetos concretos internamente.

Padrão GoF utilizado:

Nenhum

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Função	build_app_container	Montar dependências da aplicação principal.
Função	build_special_app_container	Montar dependências da versão especial.
Classe	OrderService	Receber dependências prontas e coordenar pedidos.
Classe	PaymentService	Receber dependências de pagamento.
Classe	ReportService	Receber dependências de relatório.
Interface	OrderRepositoryInterface	Abstrair persistência.

Antes:

```
def main():
    s = Sis()
```

```
class PedEspecial(Sis):
```

```
...
```

Depois:

```
from src.main import build_app_container, build_special_app_container
```

```
class Sis:
```

```
def __init__(self, db_path: str = "loja.db") -> None:
```

```
    container = build_app_container(db_path)
```

```
    self._repository = container.repository
```

```
    self._order_service = container.order_service
```

```
self._payment_service = container.payment_service
```

```
self._report_service = container.report_service
```

```
class PedEspecial:
```

```
def __init__(self, db_path: str = "loja.db") -> None:
```

```
    container = build_special_app_container(db_path)
```

```
    self._repository = container.repository
```

```
    self._order_service = container.order_service
```

2.10 DIP — Regras de negócio dependiam de detalhes de notificação e pagamento.

Solução adotada:

As regras de pagamento foram movidas para `PaymentService` e para estratégias de pagamento. As notificações foram movidas para `NotificationService`, `NotificationPublisher` e `NotificationObserver`.

Com isso, as regras principais deixam de depender diretamente de print, Pix, cartão, boleto, e-mail ou SMS.

Padrão GoF utilizado:

Observer, Strategy

Classes e interfaces criadas:

Tipo	Nome	Responsabilidade
Classe abstrata/interface	<code>PaymentStrategy</code>	Contrato para meios de pagamento.
Classe	<code>PaymentStrategyRegistry</code>	Localizar a estratégia de pagamento correta.
Classe	<code>PaymentService</code>	Processar pagamento usando uma estratégia.

Classe	NotificationService	Coordenar envio de notificações.
Classe	NotificationPublisher	Publicar eventos de notificação.
Classe abstrata/interface	NotificationObserver	Contrato para canais de notificação.

Antes:

```

if t == 'normal':

    print(f"Email enviado para {n}: Pedido recebido!")

elif t == 'vip':

    print(f"Email enviado para {n}: Pedido recebido!")

    print(f"SMS enviado para {n}: Pedido VIP recebido!")

elif t == 'corporativo':

    print(f"Email enviado para {n}: Pedido recebido!")

    print(f"Notificacao enviada ao gerente de conta de {n}")


if m == 'cartao':

    print("Processando pagamento com cartao...")

    print("Cartao validado!")

    self.upd_st(id, 'aprovado')

    return True

elif m == 'pix':

    print("Gerando QR Code PIX...")

    print("PIX recebido!")

    self.upd_st(id, 'aprovado')

    return True

```

```
elif m == 'boleto':  
  
    print("Gerando boleto...")  
  
    print("Boleto gerado!")  
  
    return True
```

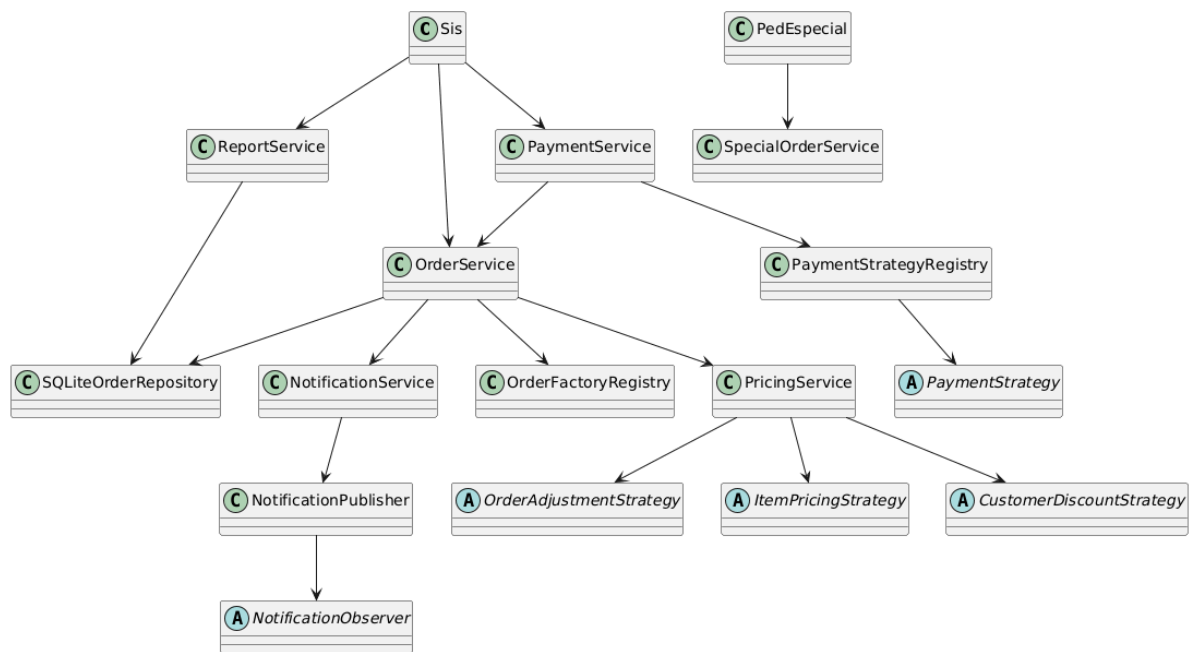
Depois:

```
def proc_pag(self, order_id: int, method: str, value: float) -> bool:  
  
    order = self._order_service.get_order_entity(order_id)  
  
    if order is None:  
  
        print("Metodo de pagamento invalido!")  
  
        return False  
  
  
  
    return self._payment_service.process_payment(  
  
        order_id=order_id,  
  
        method=method,  
  
        value=value,  
  
        order_total=order.total,  
  
    )
```

```
def add_ped(self, n: str, its: list[LegacyItemPayload], t: str) -> int:  
  
    return self._order_service.create_order(n, its, t)
```

```
def upd_st(self, order_id: int, status: str) -> None:  
  
    self._order_service.update_status(order_id, status)
```

3. (c) Diagrama UML de Classes



4. Melhorias de Clean Code

Esta seção cataloga as decisões de Clean Code tomadas durante a refatoração do sistema legado Loja Verde. Cada melhoria é listada com sua justificativa, relacionando o problema original ao código resultante.

4.1 Refatorações de Nomenclatura

O código legado adotava abreviações opacas herdadas de um contexto de desenvolvimento acelerado. A refatoração priorizou nomes que revelam intenção (Intention-Revealing Names, capítulo 2 de Clean Code).

Contexto	Nome legado	Nome refatorado + justificativa
----------	-------------	---------------------------------

Classe principal	Sis	(eliminada — responsabilidades distribuídas em serviços coesos)
Método de criação	add_ped	create_order — verbo + substantivo descrevem ação e objeto
Método de busca	get_ped	get_order / get_order_entity — distingue payload legado de entidade de domínio
Método de status	upd_st	update_status — legível sem contexto externo
Método de relatório	gerar_rel	generate (em ReportService) — classe já contextualiza o domínio
Método de cálculo	calc_tot_cli	calc_total_by_customer — parâmetro explícito no nome
Parâmetro de item	its	items_payload — tipo e propósito explícitos
Parâmetro de nome	n	name — sem ambiguidade
Parâmetro de tipo	t / tp	client_type / customer_type — consistente em todo o projeto
Campo de itens (OrderItem)	nome / p / q / tipo	name / price / quantity / item_type — vocabulário ubíquo do domínio

Campo de pedido (Order)	cli / tot / st / dt	customer / total / status / created_at — leitura sem decodificação
-------------------------	---------------------	--

4.2 Métodos Extraídos

O método `god-method` original do legado realizava consulta ao banco, cálculo, notificação e persistência em um único bloco. A refatoração decompôs esse comportamento em métodos com responsabilidade única, seguindo o princípio `Extract Method` (Fowler, Refactoring).

Extração 1 — `_build_pricing_service()` em `src/main.py`

A configuração do `PricingService` — discovery de estratégias de item, cliente e ajuste de pedido — foi extraída de `build_app_container()` para `_build_pricing_service()`. Isso elimina duplicação entre os dois containers (`AppContainer` e `SpecialAppContainer`) e isola a lógica de composição.

```
# Antes: lógica embutida diretamente em build_app_container()
# Depois:
def _build_pricing_service() -> PricingService:
    return PricingService(
        item_strategies=discover_instances('src.strategies', ItemPricingStrategy),
        customer_strategies=discover_instances('src.strategies', CustomerDiscountStrategy),
        order_adjustments=discover_instances('src.strategies', OrderAdjustmentStrategy),
    )
```

Extração 2 — `_calculate_item_total()` em `PricingService`

O loop de cálculo por item foi extraído de `calculate_total()` para `_calculate_item_total()`. O método público `calculate_total()` orquestra o pipeline completo (itens → ajustes → desconto por cliente) sem misturar lógica de iteração com lógica de despacho de estratégia.

```
def _calculate_item_total(self, item: OrderItem) -> float:
```



```

for strategy in self._item_strategies:
    if strategy.supports(item):
        return strategy.calculate(item)
raise ValueError(f'Unsupported item type: {item.item_type}')

```

Extração 3 — `_apply_customer_discount()` em `PricingService`

A aplicação do desconto por tipo de cliente foi extraída para `_apply_customer_discount()`. Antes, a lógica estava embutida como cadeia `if/elif` dentro do método de cálculo total, tornando impossível testar o desconto de forma isolada.

```

def _apply_customer_discount(self, customer_type: str, total: float) -> float:
    for strategy in self._customer_strategies:
        if strategy.supports(customer_type):
            return strategy.apply(total)
    return total

```

Extração 4 — `_row_to_order()` e `_normalize_item()` em `SQLiteOrderRepository`

A conversão de tupla de banco de dados para entidade `Order` foi extraída para `_row_to_order()`. Dentro dela, a normalização de cada item `raw` foi extraída para `_normalize_item()`. Sem essa separação, o método `get_by_id()` acumulava deserialização, casting e construção de entidade.

```

def _row_to_order(self, row: tuple[object, ...]) -> Order:
    raw_items = json.loads(str(row[2]))
    payload_items = [self._normalize_item(item) for item in raw_items]
    return Order(...)

def _normalize_item(self, raw_item: object) -> LegacyItemPayload:
    if not isinstance(raw_item, dict):
        raise ValueError('Stored item must be a dictionary')
    return {'nome': str(raw_item['nome']), 'p': float(raw_item['p']), ...}

```

4.3 Abstrações Criadas

A refatoração introduziu quatro famílias de abstrações que eliminaram dependências diretas de concretos e permitiram extensão por composição.

Abstração (ABC)	Arquivo	Propósito
OrderRepositoryInterface	interfaces/order_repository_interface.py	Isola SQLite do restante do sistema; permite trocar storage sem tocar em serviços
ItemPricingStrategy	strategies/base.py	Algoritmo de precificação por tipo de item; suporta prioridade e descoberta automática
CustomerDiscountStrategy	strategies/base.py	Desconto por perfil de cliente; supports() permite coexistência sem condicional
OrderAdjustmentStrategy	strategies/base.py	Ajuste global do pedido (ex: desconto por volume) aplicado após cálculo de itens
PaymentStrategy	strategies/base.py	Método de pagamento plugável; required_amount() com default = total (hook method)

NotificationObserver	observers/base.py	Canal de notificação desacoplado; supports() filtra por tipo de cliente
OrderFactory	factories/order_factory.py	Criação de pedido por tipo de cliente; FallbackOrderFactory garante cobertura total
StatusUpdater (Protocol)	services/payment_service.py	Duck-typing estrutural: PaymentService depende somente de update_status(), não de OrderService inteiro

O uso de Protocol (StatusUpdater) merece destaque: em vez de criar uma interface ABC artificial que OrderService precisaria herdar, usou-se tipagem estrutural do Python.

PaymentService descreve apenas o contrato mínimo que precisa — um único método — evitando acoplamento desnecessário a toda a classe OrderService.

4.4 Eliminação de Duplicação

O código legado continha pelo menos cinco focos de duplicação estrutural. Cada um foi eliminado por uma técnica diferente.

a. Cadeias if/elif para descontos e pagamentos — eliminadas por Strategy

O legado repetia estruturalmente o padrão if tipo == 'X' → comportamento X em múltiplos contextos (desconto por item, desconto por cliente, método de pagamento). Cada Strategy encapsula exatamente um comportamento, e o despacho é feito por supports().

b. Construção de Order duplicada entre OrderService e SpecialOrderService — eliminada por FactoryRegistry

Os dois serviços criavam Order diretamente com os mesmos campos positional. Com OrderFactory, a lógica de construção está em um único ponto; SpecialOrderFactory trata o caso especial sem copiar código.

c. Publicação de notificação duplicada por canal — eliminada por Observer + Publisher

Antes, cada chamada de serviço emitia print() condicionais por tipo de cliente.

NotificationPublisher centraliza o despacho: qualquer novo canal é adicionado como observer sem alterar a lógica de publicação.

d. Configuração de PricingService duplicada nos dois containers — eliminada por _build_pricing_service()

A composição das três listas de estratégias (item, cliente, ajuste) aparecia de forma duplicada em build_app_container() e build_special_app_container(). A função auxiliar _build_pricing_service() centralizou essa configuração.

e. Deserialização de LegacyItemPayload em múltiplos pontos — centralizada em OrderItem.from_legacy()

A conversão de dicionários {'nome', 'p', 'q', 'tipo'} para campos tipados ocorria dispersa. O classmethod from_legacy() em OrderItem concentra a lógica de conversão, e to_legacy() cuida do caminho inverso, mantendo a compatibilidade com o contrato legado.

5 Extensões Implementadas

As três extensões obrigatórias foram implementadas sem modificar nenhuma classe existente, comprovando empiricamente o Princípio Aberto/Fechado (OCP). Cada extensão consiste exclusivamente na adição de novos arquivos dentro dos subpacotes extensions/.

5.1 Extensão 1 — Pagamento em Criptomoeda (CryptoPaymentStrategy)

Diff conceitual

Arquivo	Operação
src/strategies/extensions/crypto_payment.py	ADICIONADO (novo arquivo)
src/strategies/extensions/__init__.py	ADICIONADO (pacote, arquivo vazio)
Qualquer outro arquivo existente	NÃO MODIFICADO

Trecho de código

```
# src/strategies/extensions/crypto_payment.py
from src.strategies.base import PaymentStrategy

class CryptoPaymentStrategy(PaymentStrategy):
    priority = 25
    method_name = 'criptomoeda'
    auto_approve = True

    def supports(self, method: str) -> bool:
        return method == self.method_name

    def required_amount(self, order_total: float) -> float:
        return order_total * 1.02 # taxa de 2%

    def process(self, paid_amount: float, required_amount: float) -> bool:
        print('Processando pagamento em criptomoeda...')
        print('Pagamento em criptomoeda confirmado!')
        return True
```

Por que a refatoração tornou essa extensão simples

A refatoração substituiu a cadeia if/elif de métodos de pagamento pela abstração PaymentStrategy + PaymentStrategyRegistry. O registro é populado pelo mecanismo de

descoberta automática (discover_instances em src/discovery.py), que percorre recursivamente o pacote src.strategies buscando subclasses concretas de PaymentStrategy.

Ao criar CryptoPaymentStrategy no subpacote extensions/, ela é descoberta e instanciada automaticamente na próxima execução, sem alterar PaymentService, PaymentStrategyRegistry ou qualquer outro módulo existente. O atributo required_amount() sobrescrito adiciona a taxa de 2% de forma localizada, aproveitando o hook method já previsto na classe base.

5.2 Extensão 2 — Canal de Notificação WhatsApp (WhatsAppObserver)

Diff conceitual

Arquivo	Operação
src/observers/extensions/whatsapp_observer.py	ADICIONADO (novo arquivo)
src/observers/extensions/__init__.py	ADICIONADO (pacote, arquivo vazio)
Qualquer outro arquivo existente	NÃO MODIFICADO

Trecho de código

```
# src/observers/extensions/whatsapp_observer.py

from src.models.order import Order
from src.observers.base import NotificationObserver

class WhatsAppObserver(NotificationObserver):
    priority = 15

    def supports(self, customer_type: str) -> bool:
        return True # abrange todos os tipos de cliente

    def on_order_received(self, order: Order) -> None:
        print(f'WhatsApp enviado para {order.customer.name}: Pedido recebido!')
```

```
def on_status_changed(self, order: Order, status: str) -> None:
    if status == 'aprovado':
        print(f'WhatsApp enviado para {order.customer.name}: Pedido aprovado!')
    elif status == 'enviado':
        print(f'WhatsApp enviado para {order.customer.name}: Pedido enviado!')
    elif status == 'entregue':
        print(f'WhatsApp enviado para {order.customer.name}: Pedido entregue!')
```

Por que a refatoração tornou essa extensão simples

O código legado emitia notificações como `print()` condicionais embutidos diretamente dentro dos fluxos de negócio (`create_order`, `update_status`). Adicionar um novo canal exigiria localizar e modificar cada um desses pontos, correndo risco de regressão.

Após a refatoração, `NotificationPublisher` itera sobre todos os observers registrados e chama `on_order_received()` ou `on_status_changed()` para cada um que retornar `True` em `supports()`. Ao adicionar `WhatsAppObserver` no subpacote `extensions/`, o mecanismo de descoberta o inclui automaticamente na lista de observers. O atributo `priority = 15` posiciona o `WhatsApp` após o `Email` (`priority 10`) e antes do `SMS` (`priority 20`), controlando a ordem de entrega sem tocar em nenhum observer existente.

5.3 Extensão 3 — Desconto por Volume (`VolumeDiscountStrategy`)

Diff conceitual

Arquivo	Operação
<code>src/strategies/extensions/volume_discount.py</code>	ADICIONADO (novo arquivo)
<code>src/strategies/extensions/__init__.py</code>	ADICIONADO (pacote, já criado na extensão 1)
Qualquer outro arquivo existente	NÃO MODIFICADO

Trecho de código

```

# src/strategies/extensions/volume_discount.py
from collections import Counter
from collections.abc import Sequence
from src.models.order_item import OrderItem
from src.strategies.base import OrderAdjustmentStrategy

class VolumeDiscountStrategy(OrderAdjustmentStrategy):
    priority = 10

    def apply(self, items: Sequence[OrderItem], current_total: float) -> float:
        quantities = Counter(
            item.name for item in items for _ in range(item.quantity)
        )
        discount = 0.0
        for item in items:
            if quantities[item.name] >= 3:
                discount += item.price * item.quantity * 0.15
        return current_total - discount

```

Por que a refatoração tornou essa extensão simples

O legado não possuía sequer o conceito de ajuste global de pedido: descontos eram aplicados item a item, tornando impossível implementar regras que dependem da visão do pedido como um todo (como 'três ou mais unidades do mesmo produto').

A refatoração introduziu a abstração `OrderAdjustmentStrategy` e um pipeline explícito em `PricingService.calculate_total()`: primeiro calcula-se o subtotal dos itens, depois aplica-se cada `OrderAdjustmentStrategy` em sequência, e por último aplica-se o desconto por cliente. `VolumeDiscountStrategy` implementa apenas o método `apply()`, recebendo a lista de itens e o total corrente. Por ter `priority = 10`, executa antes do `NoOpOrderAdjustmentStrategy` (`priority = 1000`), garantindo ordem correta sem alterar `PricingService`.

6 Reflexão Metacognitiva

6.1 O princípio que ainda não dominei — LSP (Liskov Substitution Principle)

O LSP diz que um objeto do tipo T pode ser substituído por um objeto do tipo S (subtype de T) sem que o comportamento observável do programa se altere. Na prática, minha dificuldade está em saber quando uma herança realmente respeitará esse contrato antes de ter escrito a subclasse.

No projeto, a relação original Sis / PedEspecial violava o LSP: PedEspecial.upd_st não publicava notificações, quebrando a invariante estabelecida pelo pai. A solução adotada foi eliminar a herança e usar composição — o que funcionou bem. Mas isso gerou uma dúvida que persiste: a decisão de usar composição em vez de herança foi uma aplicação legítima do LSP, ou foi simplesmente uma esquiva? Se eu tivesse a hierarquia correta desde o início, conseguiria mantê-la sem violar o princípio?

Minha dúvida concreta é: como identificar, na fase de design, quais contratos implícitos uma subclasse precisará honrar? O enunciado clássico do LSP fala em 'comportamento observável', mas em sistemas reais esse comportamento raramente está documentado em pré/pós-condições formais. Sem essa formalização, percebo que avalio o LSP de forma retroativa — identifico a violação quando ela já causou um bug — e não de forma preventiva.

O padrão Observer usado no projeto contorna isso: cada NotificationObserver precisa apenas implementar on_order_received() e on_status_changed(), e supports() filtra quem realmente é chamado. Não há hierarquia com invariante frágil. Mas isso é uma solução de design pattern, não uma compreensão mais profunda do LSP em si.

6.2 Como usei IA neste trabalho

Prompts que funcionaram

O melhor uso foi pedir à IA para identificar violações SOLID em um trecho específico de código, listando-as por princípio. O resultado foi direto, bem estruturado e coincidiu com boa parte da análise que já havia feito mentalmente — mas a IA acrescentou dois pontos que eu não havia

notado (a ausência de separação entre operações de leitura e escrita no repositório, e o acoplamento de StatusUpdater com OrderService inteiro).

Outro prompt eficaz: pedir para revisar a consistência de nomenclatura entre os módulos antes de gerar a tabela de renomeações da seção 4.1. A IA encontrou três casos em que havia usado `customer_type` em alguns arquivos e `client_type` em outros — inconsistência real que eu havia introduzido durante a refatoração iterativa.

Prompts que falharam

Pedi uma sugestão de estrutura de diretórios para o projeto. A IA propôs separar `domain/`, `application/` e `infrastructure/` seguindo Clean Architecture estrita. A sugestão era tecnicamente válida, mas incompatível com o escopo do trabalho: o enunciado especificava a estrutura `models/`, `services/`, `repositories/`, etc. Adotar a sugestão da IA teria me afastado do que o professor pediu sem acrescentar nota.

Decisão onde discordei da sugestão da IA

A IA sugeriu que o mecanismo de descoberta automática (`discover_instances`) fosse substituído por registro explícito: cada nova Strategy ou Observer deveria ser importado e instanciado manualmente em `src/main.py`. O argumento foi que a descoberta automática por reflexão introduz 'magia' difícil de depurar e que torna o grafo de dependências implícito.

Discordei e mantive a descoberta automática por três razões concretas. Primeira: o requisito central do trabalho era demonstrar OCP — novas extensões não devem exigir modificação de arquivos existentes. Registro manual em `main.py` é, por definição, uma modificação de arquivo existente. Segunda: o mecanismo está contido em um único arquivo (`src/discovery.py`) com menos de trinta linhas, e qualquer desenvolvedor que ler os testes de extensão entenderá imediatamente que 'criar um arquivo em `extensions/` é suficiente'. Terceira: o argumento de 'magia difícil de depurar' se aplica principalmente a frameworks de injeção de dependência complexos; aqui, `discover_instances` faz apenas `pkgutil.walk_packages + __subclasses__()`, comportamento totalmente previsível e coberto por testes de unidade.